

P4-SpecTec Structured Language (SL) Specification

2026-05-16

Contents

1. Syntax	3
1.1. Identifiers	3
1.2. Types	3
1.3. Iterators	3
1.4. Variables	3
1.5. Operators	4
1.5.1. Boolean operators	4
1.5.2. Numeric operators	4
1.5.3. Comparison operators	4
1.6. Values	5
1.7. Expressions	5
1.7.1. Patterns	6
1.7.2. Paths	6
1.8. Arguments	6
1.9. Parameters	6
1.10. Instructions	7
1.11. Definitions	7
1.12. Scripts	7
2. Semantics	8
2.1. Context	8
2.1.1. Loading a context	8
2.1.2. Creating sub-contexts for iteration	9
2.2. Expressions	11
2.2.1. Boolean, number, and string literals	11
2.2.2. Variable references	12
2.2.3. Operations	12
2.2.3.1. Unary expressions	12
2.2.3.2. Binary expressions	12
2.2.3.3. Comparison expressions	13
2.2.4. Casts	14
2.2.4.1. Upcast expressions	14
2.2.4.2. Downcast expressions	16
2.2.5. Runtime type tests	18
2.2.6. Match expressions	22
2.2.7. Tuple expressions	23
2.2.8. Case expressions	24
2.2.9. Struct expressions	24
2.2.10. Option expressions	25
2.2.11. List expressions	25
2.2.12. Concatenation expressions	26
2.2.13. Membership expressions	26
2.2.14. Length expressions	27
2.2.15. Accesses	27
2.2.15.1. Dot access expressions	30
2.2.15.2. Index access expressions	30
2.2.15.3. Slice access expressions	30
2.2.16. Update expressions	31
2.2.17. Function call expressions	34

2.2.18. Iterator expressions	35
2.3. Assignments	36
2.3.1. Assigning a value to an expression	36
2.3.1.1. Variable references	37
2.3.1.2. Tuple expressions	37
2.3.1.3. Case expressions	37
2.3.1.4. Struct expressions	37
2.3.1.5. Option expressions	37
2.3.1.6. List expressions	38
2.3.1.7. Cons expressions	38
2.3.1.8. Iterator expressions.	38
2.3.2. Assigning a value to a function parameter	40
2.3.2.1. Expression parameters	41
2.3.2.2. Function parameters	41
2.4. Instructions	41
2.4.1. If instructions	43
2.4.2. If-hold instructions	44
2.4.3. Case instructions	45
2.4.4. Group instructions	46
2.4.5. Let instructions	47
2.4.6. Relation call instructions	49
2.4.7. Result instructions	53
2.4.8. Return instructions	53
2.5. Functions	53
2.5.1. Extern functions	55
2.5.2. Built-in functions	55
2.5.3. Table functions	55
2.5.4. User-defined functions.	56
2.6. Relations	57
2.6.1. Extern relations	58
2.6.2. User-defined relations	59

1. Syntax

1.1. Identifiers

```
id = text

atom = text

mixop = atom**
```

1.2. Types

```
numtyp
: NAT
| INT
;

optyp
: BOOL
| numtyp
| TEXT
;

typ
: optyp
| VAR id targ*
| TUP typ*
| ITER typ iter
| FUNC
;

deftyp
: ALIAS typ
| STRUCT typfield*
| VARIANT typcase*
;

typfield
: atom typ
;

typcase
: mixop typ*
;
```

1.3. Iterators

```
iter
: QUEST
| STAR
;
```

1.4. Variables

```
vari
```

```
: id typ iter*  
;
```

1.5. Operators

```
unop  
: boolunop  
| numunop  
;  
  
binop  
: boolbinop  
| numbinop  
;  
  
cmpop  
: polycmpop  
| numcmpop  
;
```

1.5.1. Boolean operators

```
boolunop  
: NOT  
;  
  
boolbinop  
: AND  
| OR  
| IMPL  
| EQUIV  
;
```

1.5.2. Numeric operators

```
numunop  
: PLUS  
| MINUS  
;  
  
numbinop  
: ADD  
| SUB  
| MUL  
| DIV  
| MOD  
| POW  
;
```

1.5.3. Comparison operators

```
numcmpop  
: LT  
| GT  
| LE  
| GE  
;
```

```
polycmpop
: EQ
| NE
;
```

1.6. Values

```
num
: NAT nat
| INT int
;

val
: BOOL bool
| num
| TEXT text
| STR valfield*
| INJ valcase
| TUP val*
| OPT val?
| LIST val*
| FUNC id
| EXT json
;

valfield
: atom val
;

valcase
: mixop val*
;
```

1.7. Expressions

```
exp
: BOOL bool
| num
| TEXT text
| VAR id
| UN unop exp
| BIN binop exp exp
| CMP cmpop exp exp
| UPCAST typ exp
| DOWNCAST typ exp
| SUB exp typ
| MATCH exp pattern
| TUP exp*
| INJ expcase
| STR expfield*
| OPT exp?
| LIST exp*
| CONS exp exp
| CAT exp exp
| MEM exp exp
| LEN exp
| DOT exp atom
| IDX exp exp
| SLICE exp exp exp
| UPD exp path exp
| CALL id targ* arg*
```

```

| ITER exp iterexp
;

expcase
: mixop exp*
;

expfield
: atom exp
;

iterexp
: iter vari*
;

```

1.7.1. Patterns

```

listpattern
: CONS
| FIXED nat
| NIL
;

optpattern
: SOME
| NONE
;

pattern
: INJ mixop
| listpattern
| optpattern
;

```

1.7.2. Paths

```

path
: ROOT
| IDX path exp
| SLICE path exp exp
| DOT path atom
;

```

1.8. Arguments

```

targ = typ

arg
: EXP exp
| FUN id
;

```

1.9. Parameters

```

tparam = id

param

```

```

: EXP typ exp
| FUN id : tparam* param* -> typ
;

```

1.10. Instructions

```

instr
: IF exp iterexp* block
| IFHOLD id : exp* iterexp* holdcase
| CASE exp case*
| GROUP id exp* : typ* -> typ* = block
| LET exp = exp iterinstr* block
| REL id : exp* -> exp* iterinstr* block
| RESULT typ* -> typ* : exp*
| RETURN exp
| DEBUG exp
;

iterinstr
: iter vari* vari*
;

block = instr*

holdcase
: BOTH block block
| HOLD block
| NOTHOLD block
;

guard
: BOOL bool
| CMP cmpop exp
| SUB typ
| MATCH pattern
| MEM exp
;

case
: guard block
;

```

1.11. Definitions

```

defn
: EXTTYPE id
| TYP id tparam* = deftyp
| EXTREL id exp* : typ* -> typ*
| REL id exp* : typ* -> typ* = block block?
| EXTFUNC id tparam* param* : typ
| BUILTINFUNC id tparam* param* : typ
| TABLEFUNC id param* : typ = tblrow*
| FUNC id tparam* param* : typ = block block?
;

tblrow
: exp* = exp - block
;

```

1.12. Scripts


```
script = defn*
```

2. Semantics

2.1. Context

A context holds the environments used for evaluation. It is consisted of two layers, global and local, each holding four environments for values, types, functions, and relations.

```
layer = {  
  TYP tdenv,  
  REL renv,  
  FUNC fenv,  
  VAL venv  
}
```

```
tdenv = map<id, typedef>  
renv = map<id, reldef>  
fenv = map<id, funcdef>  
venv = map<varr, val>
```

2.1.1. Loading a context

Before evaluation, a context must be loaded from the specification script.

C with script defn* loaded

1. If let $\text{defn}_h :: \text{defn}_t^*$ be defn^* :
 - a. If let EXTTYP id be defn_h :
 - i. Let C' be **C with type EXT loaded as id**.
 - ii. Return **C with script defn_t* loaded**.
 - b. Else if let $\text{TYP id tparam}^* \vdash \text{deftyp}$ be defn_h :
 - i. Let C' be **C with type DEF tparam* $\rightarrow$ deftyp loaded as id**.
 - ii. Return **C with script defn_t* loaded**.
 - c. Else if let $\text{EXTREL id exp}^* \vdash \text{typ}_{\text{input}}^* \rightarrow \text{typ}_{\text{output}}^*$ be defn_h :
 - i. Let C' be **C with relation EXT id loaded as id**.
 - ii. Return **C with script defn_t* loaded**.
 - d. Else if let $\text{REL id exp}^* \vdash \text{typ}_{\text{input}}^* \rightarrow \text{typ}_{\text{output}}^* \vdash \text{block block}_{\text{els}}^?$ be defn_h :
 - i. Let C' be **C with relation DEF exp* block block_{els}? loaded as id**.
 - ii. Return **C with script defn_t* loaded**.
 - e. Else if let $\text{EXTFUNC id tparam}^* \text{ param}^* \vdash \text{typ}$ be defn_h :
 - i. Let C' be **C with function EXT id loaded as id**.

- ii. Return **C' with script** defn_t^* **loaded**.
- f. Else if let **BUILTINFUNC** id $\text{tparam}^* \text{param}^* \text{typ}$ be defn_n :
 - i. Let **C'** be **C with function** **BUILTIN** id $\text{tparam}^* \text{param}^*$ **loaded as id**.
 - ii. Return **C' with script** defn_t^* **loaded**.
- g. Else if let **TABLEFUNC** id $\text{param}^* \text{typ} = \text{tblrow}^*$ be defn_n :
 - i. Let **C'** be **C with function** **TABLE** $\text{param}^* \rightarrow \text{tblrow}^*$ **loaded as id**.
 - ii. Return **C' with script** defn_t^* **loaded**.
- h. Else:
 - i. Let **FUNC** id $\text{tparam}^* \text{param}^* \text{typ} = \text{block block}_{\text{els}}^?$ be defn_n .
 - ii. Let **C'** be **C with function** **DEF** $\text{tparam}^* \text{param}^* \rightarrow \text{block block}_{\text{els}}^?$ **loaded as id**.
 - iii. Return **C' with script** defn_t^* **loaded**.
- 2. Else:
 - a. Return **C**.

C with type **typedef** **loaded as id**

1. Let $\text{tdenv}_{\text{update}}$ be **C.GLOBAL.TYP** **where id to typedef is added**.
2. Return **C** with **GLOBAL.TYP** set to $\text{tdenv}_{\text{update}}$.

C with relation **reldf** **loaded as id**

1. Let $\text{renv}_{\text{update}}$ be **C.GLOBAL.REL** **where id to reldf is added**.
2. Return **C** with **GLOBAL.REL** set to $\text{renv}_{\text{update}}$.

C with function **funcdef** **loaded as id**

1. Let $\text{fenv}_{\text{update}}$ be **C.GLOBAL.FUNC** **where id to funcdef is added**.
2. Return **C** with **GLOBAL.FUNC** set to $\text{fenv}_{\text{update}}$.

2.1.2. Creating sub-contexts for iteration

When evaluating iteration (via $?$ or $*$), sub-contexts are created for each dimension being iterated over.

The sub-context of C for optional iterators vari^*

1. Let $\text{vari}_{\text{iter}}^*$ be the list obtained by repeating:
 - Let $\text{vari}_{\text{iter}}$ be **vari iterated by QUEST**.
 for each **vari** in vari^*
2. Let $\text{val}^{""?}$ be the list obtained by repeating:

- Let $val^{m?}$ be the value of $vari_{iter}$ in C .

for each $vari_{iter}$ in $vari_{iter}^*$

3. If $val^{m?}$ is defined, for all $val^{m?}$ in $val^{m?}$:

- a. Let val^{m*} be the list obtained by repeating:

- Let val^m be $val^{m?}$.

for each $val^{m?}$ in $val^{m?}$

- b. Check that val^m matches pattern $OPT \%$, for all val^m in val^{m*} .

- c. Let $val^{l?}$ be the list obtained by repeating:

- Let $OPT \ val^{l?}$ be val^m .

for each val^m in val^{m*}

- d. Check that $val^{l?}$ is defined, for all $val^{l?}$ in $val^{l?}$.

- e. Let val^{l*} be the list obtained by repeating:

- Let val be $val^{l?}$.

for each $val^{l?}$ in $val^{l?}$

- f. Let ctx be C where each of $vari^*$ to each of val^* added locally.

- g. Return ctx .

4. If $OPT \cdot$ is equal to the value of $vari_{iter}$ in C , for all $vari_{iter}$ in $vari_{iter}^*$:

- a. Return $\cdot$.

The sub-contexts of C for list iterators $vari^*$

1. Let $vari_{iter}^*$ be the list obtained by repeating:

- Let $vari_{iter}$ be $vari$ iterated by $STAR$.

for each $vari$ in $vari^*$

2. Let $val^{l?}$ be the list obtained by repeating:

- Let $val^{m?}$ be the value of $vari_{iter}$ in C .

for each $vari_{iter}$ in $vari_{iter}^*$

3. Check that $val^{m?}$ is defined, for all $val^{m?}$ in $val^{m?}$.

4. Let val^{l*} be the list obtained by repeating:

- Let val^l be $val^{m?}$.

for each $val^{m?}$ in $val^{m?}$

5. Check that val^l matches pattern $LIST \%$, for all val^l in val^{l*} .

6. Let val^{**} be the list obtained by repeating:

- Let $LIST\ val^*$ be val' .

for each val' in val^*

7. Let val_{trans}^{**} be $\$transpose_<val>(val^{**})$.

8. Let C^* be the list obtained by repeating:

- Let C' be $\$sub_list'(C, vari^*, val_{trans}^*)$.

for each val_{trans}^* in val_{trans}^{**}

9. Return C^* .

[[sub_list']]

$\$sub_list'(C, vari^*, val^*)$

1. If $vari^*$ is an empty list:

- a. Check that val^* is an empty list.
- b. Return C .

2. Else:

- a. Let $vari_h :: vari_t^*$ be $vari^*$.
- b. Check that val^* is a non-empty list.
- c. Let $val_h :: val_t^*$ be val^* .
- d. Let C' be C where $vari_h$ to val_h added locally.
- e. Return $\$sub_list'(C', vari_t^*, val_t^*)$.

2.2. Expressions

An expression is evaluated with the following relation:

$Eval_exp$:

- Evaluating exp under C :
- Result in $res<val>$.

If no rule was successfully applied, the evaluation fails:

2.2.1. Boolean, number, and string literals

Evaluating exp under C :

1. If let **BOOL** *b* be *exp*":
 - a. Result in **OK** **BOOL** *b*.
2. Else if let **num** be *exp*":
 - a. Result in **OK** **num**.
3. Else:
 - a. Let **TEXT** *t* be *exp*".
 - b. Result in **OK** **TEXT** *t*.

2.2.2. Variable references

Evaluating **VAR** *id* under *C*:

1. Let *val* be ! the value of *id* · in *C*.
2. Result in **OK** *val*.

2.2.3. Operations

2.2.3.1. Unary expressions

Evaluating **UN** *unop* *exp* under *C*:

1. If *unop* is equal to **NOT**:
 - a. Let *res<val>* be
 - the result of evaluating *exp* under *C*.
 - b. Check that *res<val>* matches pattern **OK** %.
 - c. Let **OK** *val* be *res<val>*.
 - d. Check that *val* matches pattern **BOOL** %.
 - e. Let **BOOL** *b* be *val*.
 - f. Result in **OK** **BOOL** ~*b*.
2. If let **num***unop* be *unop*:
 - a. Let *res<val>* be
 - the result of evaluating *exp* under *C*.
 - b. Check that *res<val>* matches pattern **OK** %.
 - c. Let **OK** *val* be *res<val>*.
 - d. Check that *val* has type **num**.
 - e. Let **num** be *val*.
 - f. Let *val_{res}* be \$*unop*_number(*numunop*, *num*).
 - g. Result in **OK** *val_{res}*.

2.2.3.2. Binary expressions

Evaluating BIN binop exp_l exp_r under C:

1. If let boolbinop be binop:
 - a. Let res<val> be
 - the result of evaluating exp_l under C.
 - b. Check that res<val> matches pattern OK %.
 - c. Let OK val be res<val>.
 - d. Check that val matches pattern BOOL %.
 - e. Let BOOL b_l be val.
 - f. Let res<val>' be
 - the result of evaluating exp_r under C.
 - g. Check that res<val>' matches pattern OK %.
 - h. Let OK val' be res<val>'.
 - i. Check that val' matches pattern BOOL %.
 - j. Let BOOL b_r be val'.
 - k. Let b_{res} be \$binop_bool(boolbinop, b_l, b_r).
 - l. Result in OK BOOL b_{res}.
2. Else:
 - a. Let numbinop be binop.
 - b. Let res<val> be
 - the result of evaluating exp_l under C.
 - c. Check that res<val> matches pattern OK %.
 - d. Let OK val be res<val>.
 - e. Check that val has type num.
 - f. Let num_l be val.
 - g. Let res<val>' be
 - the result of evaluating exp_r under C.
 - h. Check that res<val>' matches pattern OK %.
 - i. Let OK val' be res<val>'.
 - j. Check that val' has type num.
 - k. Let num_r be val'.
 - l. Let val_{res} be \$binop_number(numbinop, num_l, num_r).
 - m. Result in OK val_{res}.

2.2.3.3. Comparison expressions

Evaluating CMP cmpop exp_l exp_r under C:

1. If let `polycmpop` be `cmpop`:
 - a. Let `res<val>` be
 - the result of evaluating `expi` under `C`.
 - b. Check that `res<val>` matches pattern `OK %`.
 - c. Let `OK vali` be `res<val>`.
 - d. Let `res<val>'` be
 - the result of evaluating `expr` under `C`.
 - e. Check that `res<val>'` matches pattern `OK %`.
 - f. Let `OK valr` be `res<val>'`.
 - g. Let `bres` be `$cmpop_poly(polycmpop, vali, valr)`.
 - h. Result in `OK BOOL bres`.
2. Else:
 - a. Let `numcmpop` be `cmpop`.
 - b. Let `res<val>` be
 - the result of evaluating `expi` under `C`.
 - c. Check that `res<val>` matches pattern `OK %`.
 - d. Let `OK val` be `res<val>`.
 - e. Check that `val` has type `num`.
 - f. Let `numi` be `val`.
 - g. Let `res<val>'` be
 - the result of evaluating `expr` under `C`.
 - h. Check that `res<val>'` matches pattern `OK %`.
 - i. Let `OK val'` be `res<val>'`.
 - j. Check that `val'` has type `num`.
 - k. Let `numr` be `val'`.
 - l. Let `bres` be `$cmpop_number(numcmpop, numi, numr)`.
 - m. Result in `OK BOOL bres`.

2.2.4. Casts

2.2.4.1. Upcast expressions

Evaluating `UPCAST typ exp` under `C`:

1. Let `res<val>` be
 - the result of evaluating `exp` under `C`.
2. Check that `res<val>` matches pattern `OK %`.
3. Let `OK val` be `res<val>`.
4. Result in `val upcasted to typ` under `C`.

Upcasting is defined as follows:

val' upcasted to typ'' under C

1. If let $numtyp$ be typ'' :
 - a. Check that $numtyp$ is INT .
 - b. If let num be val' :
 - i. If let $INT\ i$ be num :
 - A. Return $OK\ INT\ i$.
 - ii. Else:
 - A. Let $NAT\ n$ be num .
 - B. Return $OK\ INT\ n$.
 - c. Else:
 - i. Return $FAIL$.
2. Else if let $VAR\ id\ targ^*$ be typ'' :
 - a. Let $typdef$ be ! the type definition of id in C .
 - b. Check that $typdef$ matches pattern $DEF\ \% \rightarrow \%$.
 - c. Let $DEF\ tparam^* \rightarrow deftyp$ be $typdef$.
 - d. Check that $deftyp$ matches pattern $ALIAS\ \%$.
 - e. Let $ALIAS\ typ$ be $deftyp$.
 - f. Check that the length of $targ^*$ is equal to the length of $tparam^*$.
 - g. Let $theta$ be $\{(tparam^* : targ^*)\}$.
 - h. Let typ' be typ substituted by $theta$.
 - i. Return val' upcasted to typ' under C .
3. Else if let $TUP\ typ^*$ be typ'' :
 - a. If let $TUP\ val^*$ be val' :
 - i. Check that the length of typ^* is equal to the length of val^* .
 - ii. Let $res<val>^*$ be the list obtained by repeating:
 - Let $res<val>$ be val upcasted to typ under C .for each typ in typ^* and val in val^*
 - iii. Check that $res<val>$ matches pattern $OK\ \%$, for all $res<val>$ in $res<val>^*$.
 - iv. Let val_{upcast}^* be the list obtained by repeating:
 - Let $OK\ val_{upcast}$ be $res<val>$.for each $res<val>$ in $res<val>^*$
 - v. Return $OK\ TUP\ val_{upcast}^*$.
 - b. If val' is not a tuple:
 - i. Return $FAIL$.

4. Else:

a. Let `ITER typ iter` be `typ`".

b. If `iter` is `QUEST`:

i. Check that `val'` matches pattern `OPT %`.

ii. Let `OPT val'` be `val'`.

iii. Let `res<val>*` be the list obtained by repeating:

- Let `res<val>` be `val` **upcasted to `typ` under `C`**.

for each `val` in `val*`

iv. Check that `res<val>` matches pattern `OK %`, for all `res<val>` in `res<val>*`.

v. Let `valupcast*` be the list obtained by repeating:

- Let `OK valupcast` be `res<val>`.

for each `res<val>` in `res<val>*`

vi. Return `OK OPT valupcast?`.

c. Else:

i. Check that `val'` matches pattern `LIST %`.

ii. Let `LIST val*` be `val'`.

iii. Let `res<val>*` be the list obtained by repeating:

- Let `res<val>` be `val` **upcasted to `typ` under `C`**.

for each `val` in `val*`

iv. Check that `res<val>` matches pattern `OK %`, for all `res<val>` in `res<val>*`.

v. Let `valupcast*` be the list obtained by repeating:

- Let `OK valupcast` be `res<val>`.

for each `res<val>` in `res<val>*`

vi. Return `OK LIST valupcast*`.

5. Otherwise:

a. Return `OK val'`.

2.2.4.2. Downcast expressions

`val'` **downcasted to `typ` under `C`**

1. If let `numtyp` be `typ`":

a. Check that `numtyp` is `NAT`.

b. If let `num` be `val`":

i. If let `NAT n` be `num`:

- A. Return OK NAT n .
 - ii. Else:
 - A. Let INT i be num .
 - B. Check that i has type nat .
 - C. Let n be i .
 - D. Return OK NAT n .
 - c. Else:
 - i. Return FAIL.
- 2. Else if let VAR id targ^* be typ'' :
 - a. Let typdef be ! the type definition of id in C .
 - b. Check that typdef matches pattern DEF $\% \rightarrow \%$.
 - c. Let DEF $\text{tparam}^* \rightarrow \text{deftyp}$ be typdef .
 - d. Check that deftyp matches pattern ALIAS $\%$.
 - e. Let ALIAS typ be deftyp .
 - f. Check that the length of targ^* is equal to the length of tparam^* .
 - g. Let theta be $\{ (\text{tparam} : \text{targ})^* \}$.
 - h. Let typ' be typ substituted by theta .
 - i. Return val' downcasted to typ' under C .
- 3. Else if let TUP typ^* be typ'' :
 - a. If let TUP val^* be val' :
 - i. Check that the length of typ^* is equal to the length of val^* .
 - ii. Let $\text{res}<\text{val}>^*$ be the list obtained by repeating:
 - Let $\text{res}<\text{val}>$ be val downcasted to typ under C .
 - for each typ in typ^* and val in val^*
 - iii. Check that $\text{res}<\text{val}>$ matches pattern OK $\%$, for all $\text{res}<\text{val}>$ in $\text{res}<\text{val}>^*$.
 - iv. Let $\text{val}_{\text{downcast}}^*$ be the list obtained by repeating:
 - Let OK $\text{val}_{\text{downcast}}$ be $\text{res}<\text{val}>$.
 - for each $\text{res}<\text{val}>$ in $\text{res}<\text{val}>^*$
 - v. Return OK TUP $\text{val}_{\text{downcast}}^*$.
 - b. If val' is not a tuple:
 - i. Return FAIL.
- 4. Else:
 - a. Let ITER typ iter be typ'' .
 - b. If iter is QUEST:
 - i. Check that val' matches pattern OPT $\%$.
 - ii. Let OPT $\text{val}^?$ be val' .

- iii. Let $\text{res}\langle\text{val}\rangle^*$ be the list obtained by repeating:
 - Let $\text{res}\langle\text{val}\rangle$ be val downcasted to typ under C .
 for each val in val^*
- iv. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$, for all $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$.
- v. Let $\text{val}_{\text{downcast}}^*$ be the list obtained by repeating:
 - Let $\text{OK } \text{val}_{\text{downcast}}$ be $\text{res}\langle\text{val}\rangle$.
 for each $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$
- vi. Return $\text{OK OPT } \text{val}_{\text{downcast}}^?$.
- c. Else:
 - i. Check that val' matches pattern $\text{LIST } \%$.
 - ii. Let $\text{LIST } \text{val}^*$ be val' .
 - iii. Let $\text{res}\langle\text{val}\rangle^*$ be the list obtained by repeating:
 - Let $\text{res}\langle\text{val}\rangle$ be val downcasted to typ under C .
 for each val in val^*
 - iv. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$, for all $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$.
 - v. Let $\text{val}_{\text{downcast}}^*$ be the list obtained by repeating:
 - Let $\text{OK } \text{val}_{\text{downcast}}$ be $\text{res}\langle\text{val}\rangle$.
 for each $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$
 - vi. Return $\text{OK LIST } \text{val}_{\text{downcast}}^*$.
- 5. Otherwise:
 - a. Return $\text{OK } \text{val}'$.

Downcasting is defined as follows:

Evaluating $\text{DOWNCAST } \text{typ } \text{exp}$ under C :

1. Let $\text{res}\langle\text{val}\rangle$ be
 - the result of evaluating exp under C .
2. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$.
3. Let $\text{OK } \text{val}$ be $\text{res}\langle\text{val}\rangle$.
4. Result in val downcasted to typ under C .

2.2.5. Runtime type tests

Evaluating $\text{SUB } \text{exp } \text{typ}$ under C :

1. Let $res\langle val \rangle$ be
 - the result of evaluating exp under C .
2. Check that $res\langle val \rangle$ matches pattern $OK \%$.
3. Let $OK\ val$ be $res\langle val \rangle$.
4. Let b be val matches type typ under C .
5. Result in $OK\ BOOL\ b$.

Runtime type tests are defined as follows:

val'' matches type typ' under C

1. If let $optyp$ be typ' :
 - a. If $optyp$ is $BOOL$:
 - i. Check that val'' matches pattern $BOOL \%$.
 - ii. Return $true$.
 - b. Else:
 - i. Check that val'' matches pattern $TEXT \%$.
 - ii. Return $true$.
2. Else if let $VAR\ id\ targ^*$ be typ' :
 - a. If val'' matches pattern $EXT \%$:
 - i. Check that EXT is equal to the type definition of id in C .
 - ii. Return $true$.
 - b. Let $typedef^?$ be the type definition of id in C .
 - c. If let $typedef$ be $typedef^?$:
 - i. Check that $typedef$ matches pattern $DEF \% \rightarrow \%$.
 - ii. Let $DEF\ tparam^* \rightarrow deftyp$ be $typedef$.
 - iii. Check that $deftyp$ matches pattern $ALIAS \%$.
 - iv. Let $ALIAS\ typ$ be $deftyp$.
 - v. Check that the length of $targ^*$ is equal to the length of $tparam^*$.
 - vi. Let $theta$ be $\{ (tparam\ `: targ)^* \}$.
 - vii. Let typ_{subst} be typ substituted by $theta$.
 - viii. Return val'' matches type typ_{subst} under C .
 - d. If let $INJ\ mixop\ val^*$ be val'' :
 - i. Let $typedef$ be ! the type definition of id in C .
 - ii. Check that $typedef$ matches pattern $DEF \% \rightarrow \%$.
 - iii. Let $DEF\ tparam^* \rightarrow deftyp$ be $typedef$.
 - iv. Check that $deftyp$ matches pattern $VARIANT \%$.
 - v. Let $VARIANT\ typcase^*$ be $deftyp$.
 - vi. Check that the length of $targ^*$ is equal to the length of $tparam^*$.

- vii. Let θ be $\{(tparam : targ)^*\}$.
 - viii. Let $mixop_{case}^*$ be the list and typ_{case}^{**} be the list obtained by repeating:
 - Let $mixop_{case} typ_{case}^*$ be typ_{case} .
 for each typ_{case} in typ_{case}^*
 - ix. Check that $mixop$ is in $mixop_{case}^*$.
 - x. Let $typ_{case_found}^*$ be ! the value associated to $mixop$ in $(mixop_{case}, typ_{case}^*)^*$.
 - xi. Let $typ_{case_found_subst}^*$ be $(\$subst_typ(\theta, typ_{case_found}))^*$.
 - xii. Check that the length of $typ_{case_found_subst}^*$ is equal to the length of val^* .
 - xiii. Let $bool^*$ be the list and $bool^{**}$ be the list obtained by repeating:
 - Let $bool$ be val matches type $typ_{case_found_subst}$ under C .
 for each $typ_{case_found_subst}$ in $typ_{case_found_subst}^*$ and val in val^*
 - xiv. Return all $bool^*$ are true.
- e. Else:
- i. Let $STR valfield^*$ be val .
 - ii. Let $typedef$ be ! the type definition of id in C .
 - iii. Check that $typedef$ matches pattern $DEF \% \rightarrow \%$.
 - iv. Let $DEF tparam^* \rightarrow deftyp$ be $typedef$.
 - v. Check that $deftyp$ matches pattern $STRUCT \%$.
 - vi. Let $STRUCT typfield^*$ be $deftyp$.
 - vii. Let $atom_{typ}^*$ be the list and typ_{field}^* be the list obtained by repeating:
 - Let $atom_{typ} typ_{field}$ be typ_{field} .
 for each typ_{field} in typ_{field}^*
 - viii. Let $atom_{val}^*$ be the list and val_{field}^* be the list obtained by repeating:
 - Let $atom_{val} val_{field}$ be val_{field} .
 for each val_{field} in val_{field}^*
 - ix. Check that the length of $targ^*$ is equal to the length of $tparam^*$.
 - x. Let θ be $\{(tparam : targ)^*\}$.
 - xi. Let $typ_{field_subst}^*$ be the list obtained by repeating:
 - Let typ_{field_subst} be typ_{field} substituted by θ .
 for each typ_{field} in typ_{field}^*
 - xii. Check that the length of $atom_{typ}^*$ is equal to the length of $atom_{val}^*$ and the length of $atom_{val}^*$ is equal to the length of $typ_{field_subst}^*$ and the length of $typ_{field_subst}^*$ is equal to the length of val_{field}^* .
 - xiii. Let $bool$ be val_{field} matches type typ_{field_subst} under C .

- xiv. Let b^* be the list obtained by repeating:
- Let b be atom_{typ} is equal to atom_{val} and bool .
- for each atom_{typ} in $\text{atom}_{\text{typ}}^*$, atom_{val} in $\text{atom}_{\text{val}}^*$, $\text{typ}_{\text{field_subst}}$ in $\text{typ}_{\text{field_subst}}^*$, and $\text{val}_{\text{field}}$ in $\text{val}_{\text{field}}^*$
- xv. Return $\text{all } b^* \text{ are true}$.
3. Else if let $\text{TUP } \text{typ}^*$ be typ' :
- Check that val'' matches pattern $\text{TUP } \%$.
 - Let $\text{TUP } \text{val}^*$ be val'' .
 - Check that the length of typ^* is equal to the length of val^* .
 - Let bool^* be the list and bool^{**} be the list obtained by repeating:
 - Let bool be val matches type typ under C .

for each typ in typ^* and val in val^*
- Return $\text{all } \text{bool}^* \text{ are true}$.
4. Else:
- Let $\text{ITER } \text{typ } \text{iter}$ be typ' .
 - If iter is QUEST :
 - If let $\text{OPT } \text{val}^?$ be val'' :
 - Check that $\text{val}^?$ is defined.
 - Let val be $\text{val}^?$.
 - Return val matches type typ under C .
 - If val'' is equal to $\text{OPT} :$: return true .
 - Else:
 - Check that val'' matches pattern $\text{LIST } \%$.
 - Let $\text{LIST } \text{val}^*$ be val'' .
 - Let bool^* be the list obtained by repeating:
 - Let bool be val matches type typ under C .

for each val in val^*
 - Return $\text{all } \text{bool}^* \text{ are true}$.
5. If let numtyp be typ' :
- If numtyp is NAT :
 - Check that val'' has type num .
 - Let num be val'' .
 - If num matches pattern $\text{NAT } \%$: return true .
 - Else:
 - Let $\text{INT } i$ be num .
 - Return i is greater than or equal to 0 .

- b. Else:
 - i. Check that `val` has type `num`.
 - ii. Return `true`.
- 6. Otherwise: return `false`.

Each of `val`* matches each of type `typ`* under `C`

1. If the length of `typ`* is equal to the length of `val`*:
 - a. Let `bool`* be the list and `bool`** be the list obtained by repeating:
 - Let `bool` be `val` matches type `typ` under `C`.
 for each `typ` in `typ`* and `val` in `val`*
 - b. Return `all bool`* are `true`.
2. Otherwise: return `false`.

2.2.6. Match expressions

Evaluating `MATCH exp pattern` under `C`:

1. If let `INJ mixopp` be `pattern`:
 - a. Let `res<val>` be
 - the result of `evaluating exp` under `C`.
 - b. Check that `res<val>` matches pattern `OK %`.
 - c. Let `OK val` be `res<val>`.
 - d. Check that `val` matches pattern `INJ %`.
 - e. Let `INJ mixopv _` be `val`.
 - f. Result in `OK BOOL mixopp = mixopv`.
2. Else:
 - a. Let `listpattern` be `pattern`.
 - b. Check that `listpattern` matches pattern `FIXED %`.
 - c. Let `FIXED n` be `listpattern`.
 - d. Let `res<val>` be
 - the result of `evaluating exp` under `C`.
 - e. Check that `res<val>` matches pattern `OK %`.
 - f. Let `OK val'` be `res<val>`.
 - g. Check that `val'` matches pattern `LIST %`.
 - h. Let `LIST val*` be `val'`.

- i. Result in OK BOOL the length of $val^* = n$.
3. If **pattern** is equal to **CONS**:
 - a. Let $res<val>$ be
 - the result of **evaluating** exp **under** C .
 - b. Check that $res<val>$ matches pattern OK %.
 - c. Let OK val' be $res<val>$.
 - d. Check that val' matches pattern LIST %.
 - e. Let LIST val^* be val' .
 - f. Result in OK BOOL the length of $val^* > 0$.
4. Else if **pattern** is equal to **NIL**:
 - a. Let $res<val>$ be
 - the result of **evaluating** exp **under** C .
 - b. Check that $res<val>$ matches pattern OK %.
 - c. Let OK val' be $res<val>$.
 - d. Check that val' matches pattern LIST %.
 - e. Let LIST val^* be val' .
 - f. Result in OK BOOL the length of $val^* = 0$.
5. Else if **pattern** is equal to **SOME**:
 - a. Let $res<val>$ be
 - the result of **evaluating** exp **under** C .
 - b. Check that $res<val>$ matches pattern OK %.
 - c. Let OK val' be $res<val>$.
 - d. Check that val' matches pattern OPT %.
 - e. Let OPT $val^?$ be val' .
 - f. Result in OK BOOL $val^? \neq \cdot$.
6. Else:
 - a. Let $res<val>$ be
 - the result of **evaluating** exp **under** C .
 - b. Check that $res<val>$ matches pattern OK %.
 - c. Let OK val' be $res<val>$.
 - d. Check that val' matches pattern OPT %.
 - e. Let OPT $val^?$ be val' .
 - f. Result in OK BOOL $val^? = \cdot$.

2.2.7. Tuple expressions

Evaluating TUP exp^* **under** C :

1. Let $\text{res}\langle\text{val}\rangle^*$ be the list obtained by repeating:
 - Let $\text{res}\langle\text{val}\rangle$ be
 - the result of evaluating exp under C .
 for each exp in exp^*
2. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$, for all $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$.
3. Let val^* be the list obtained by repeating:
 - Let OK val be $\text{res}\langle\text{val}\rangle$.
 for each $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$
4. Result in OK TUP val^* .

2.2.8. Case expressions

Evaluating INJ mixop exp^* under C :

1. Let $\text{res}\langle\text{val}\rangle^*$ be the list obtained by repeating:
 - Let $\text{res}\langle\text{val}\rangle$ be
 - the result of evaluating exp under C .
 for each exp in exp^*
2. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$, for all $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$.
3. Let val^* be the list obtained by repeating:
 - Let OK val be $\text{res}\langle\text{val}\rangle$.
 for each $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$
4. Result in $\text{OK INJ mixop val}^*$.

2.2.9. Struct expressions

Evaluating STR expfield^* under C :

1. Let atom^* be the list and exp^* be the list obtained by repeating:
 - Let atom exp be expfield .
 for each expfield in expfield^*
2. Let $\text{res}\langle\text{val}\rangle^*$ be the list obtained by repeating:
 - Let $\text{res}\langle\text{val}\rangle$ be
 - the result of evaluating exp under C .

for each exp in exp^*

3. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$, for all $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$.

4. Let val^* be the list obtained by repeating:

- Let OK val be $\text{res}\langle\text{val}\rangle$.

for each $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$

5. Let valfield^* be the list obtained by repeating:

- Let valfield be atom val .

for each atom in atom^* and val in val^*

6. Result in OK STR valfield^* .

2.2.10. Option expressions

Evaluating $\text{OPT exp}^?$ under C :

1. If let exp be $\text{exp}^?$:

a. Let $\text{res}\langle\text{val}\rangle$ be

- the result of evaluating exp under C .

b. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$.

c. Let OK val be $\text{res}\langle\text{val}\rangle$.

d. Result in OK OPT val .

2. Else:

a. Result in $\text{OK OPT } \cdot$.

2.2.11. List expressions

Evaluating LIST exp^* under C :

1. Let $\text{res}\langle\text{val}\rangle^*$ be the list obtained by repeating:

- Let $\text{res}\langle\text{val}\rangle$ be

- the result of evaluating exp under C .

for each exp in exp^*

2. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$, for all $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$.

3. Let val^* be the list obtained by repeating:

- Let OK val be $\text{res}\langle\text{val}\rangle$.

for each $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$

4. Result in OK LIST val^{*}.

2.2.12. Concatenation expressions

Evaluating CAT exp_i exp_r under C:

1. Let res<val> be
 - the result of evaluating exp_i under C.
2. Check that res<val> matches pattern OK %.
3. Let OK val be res<val>.
4. If let TEXT text_i be val:
 - a. Let res<val>' be
 - the result of evaluating exp_r under C.
 - b. Check that res<val>' matches pattern OK %.
 - c. Let OK val' be res<val>'.
 - d. Check that val' matches pattern TEXT %.
 - e. Let TEXT text_r be val'.
 - f. Result in OK TEXT text_i ++ text_r.
5. Else:
 - a. Let LIST val_i^{*} be val.
 - b. Let res<val>' be
 - the result of evaluating exp_r under C.
 - c. Check that res<val>' matches pattern OK %.
 - d. Let OK val' be res<val>'.
 - e. Check that val' matches pattern LIST %.
 - f. Let LIST val_r^{*} be val'.
 - g. Result in OK LIST val_i^{*} ++ val_r^{*}.

2.2.13. Membership expressions

Evaluating MEM exp_e exp_s under C:

1. Let res<val> be
 - the result of evaluating exp_e under C.
2. Check that res<val> matches pattern OK %.
3. Let OK val_e be res<val>.
4. Let res<val>' be
 - the result of evaluating exp_s under C.
5. Check that res<val>' matches pattern OK %.

6. Let **OK val'** be **res<val>'**.
7. Check that **val'** matches pattern **LIST %**.
8. Let **LIST val*** be **val'**.
9. Let **bool** be **some of (val_e = val)* is true**.
10. Result in **OK BOOL bool**.

2.2.14. Length expressions

Evaluating **LEN exp under C**:

1. Let **res<val>** be
 - the result of **evaluating exp under C**.
2. Check that **res<val>** matches pattern **OK %**.
3. Let **OK val'** be **res<val>**.
4. If let **TEXT text** be **val'**:
 - a. Result in **OK NAT the length of text**.
5. Else:
 - a. Let **LIST val*** be **val'**.
 - b. Result in **OK NAT the length of val***.

2.2.15. Accesses

Eval_path:

- Accessing **path** of **val** under **C**:
- Result in **res<val>**.

Accessing ROOT of val_b under C:

1. Result in **OK val_b**.

Accessing DOT path atom of val_b under C:

1. Let **res<val>** be
 - the result of **accessing path of val_b under C**.
2. Check that **res<val>** matches pattern **OK %**.
3. Let **OK val'** be **res<val>**.
4. Check that **val'** matches pattern **STR %**.
5. Let **STR valfield*** be **val'**.

6. Let $\text{atom}_{\text{field}}^*$ be the list and $\text{val}_{\text{field}}^*$ be the list obtained by repeating:

- Let $\text{atom}_{\text{field}} \text{val}_{\text{field}}$ be $\text{val}_{\text{field}}$.

for each $\text{val}_{\text{field}}$ in $\text{val}_{\text{field}}^*$

7. Let val be ! the value associated to atom in $(\text{atom}_{\text{field}}, \text{val}_{\text{field}})^*$.

8. Result in OK val .

Accessing IDX path exp_i of val_b under C :

1. Let $\text{res}<\text{val}>$ be

- the result of accessing path of val_b under C .

2. Check that $\text{res}<\text{val}>$ matches pattern OK %.

3. Let OK val' be $\text{res}<\text{val}>$.

4. If let TEXT t be val' :

a. Let $\text{res}<\text{val}>'$ be

- the result of evaluating exp_i under C .

b. Check that $\text{res}<\text{val}>'$ matches pattern OK %.

c. Let OK val be $\text{res}<\text{val}>'$.

d. Check that val has type num .

e. Let num be val .

f. Check that num matches pattern NAT %.

g. Let NAT n be num .

h. Check that n is less than the length of t .

i. Result in OK TEXT $t[n]$.

5. Else:

a. Let LIST val^* be val' .

b. Let $\text{res}<\text{val}>'$ be

- the result of evaluating exp_i under C .

c. Check that $\text{res}<\text{val}>'$ matches pattern OK %.

d. Let OK val'' be $\text{res}<\text{val}>'$.

e. Check that val'' has type num .

f. Let num be val'' .

g. Check that num matches pattern NAT %.

h. Let NAT n be num .

i. Check that n is less than the length of val^* .

j. Result in OK $\text{val}^*[n]$.

Accessing SLICE path $\text{exp}_i \text{exp}_n$ of val_b under C :

1. Let $\text{res}\langle\text{val}\rangle$ be
 - the result of **accessing** path of val_b under C .
2. Check that $\text{res}\langle\text{val}\rangle$ matches pattern OK %.
3. Let $\text{OK val}'$ be $\text{res}\langle\text{val}\rangle$.
4. If let TEXT t be val' :
 - a. Let $\text{res}\langle\text{val}\rangle'$ be
 - the result of **evaluating** exp_i under C .
 - b. Check that $\text{res}\langle\text{val}\rangle'$ matches pattern OK %.
 - c. Let OK val be $\text{res}\langle\text{val}\rangle'$.
 - d. Check that val has type num .
 - e. Let num be val .
 - f. Check that num matches pattern NAT %.
 - g. Let $\text{NAT } n_i$ be num .
 - h. Let $\text{res}\langle\text{val}\rangle''$ be
 - the result of **evaluating** exp_n under C .
 - i. Check that $\text{res}\langle\text{val}\rangle''$ matches pattern OK %.
 - j. Let $\text{OK val}''$ be $\text{res}\langle\text{val}\rangle''$.
 - k. Check that val'' has type num .
 - l. Let num' be val'' .
 - m. Check that num' matches pattern NAT %.
 - n. Let $\text{NAT } n_n$ be num' .
 - o. Result in OK TEXT $t[n_i : n_n]$.
5. Else:
 - a. Let LIST val^* be val' .
 - b. Let $\text{res}\langle\text{val}\rangle'$ be
 - the result of **evaluating** exp_i under C .
 - c. Check that $\text{res}\langle\text{val}\rangle'$ matches pattern OK %.
 - d. Let $\text{OK val}''$ be $\text{res}\langle\text{val}\rangle'$.
 - e. Check that val'' has type num .
 - f. Let num be val'' .
 - g. Check that num matches pattern NAT %.
 - h. Let $\text{NAT } n_i$ be num .
 - i. Let $\text{res}\langle\text{val}\rangle''$ be
 - the result of **evaluating** exp_n under C .
 - j. Check that $\text{res}\langle\text{val}\rangle''$ matches pattern OK %.
 - k. Let $\text{OK val}'''$ be $\text{res}\langle\text{val}\rangle''$.
 - l. Check that val''' has type num .
 - m. Let num' be val''' .

- n. Check that num' matches pattern $\text{NAT } \%$.
- o. Let $\text{NAT } n_n$ be num' .
- p. Result in $\text{OK LIST val}^*[n_i : n_n]$.

2.2.15.1. Dot access expressions

Evaluating DOT exp atom under C:

1. Let $\text{res}\langle\text{val}\rangle$ be
 - the result of evaluating exp under C.
2. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$.
3. Let OK val be $\text{res}\langle\text{val}\rangle$.
4. Let valres be
 - the result of accessing DOT ROOT atom of val under C.
5. Result in valres .

2.2.15.2. Index access expressions

Evaluating IDX $\text{exp}_b \text{exp}_i$ under C:

1. Let $\text{res}\langle\text{val}\rangle$ be
 - the result of evaluating exp_b under C.
2. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$.
3. Let OK val_b be $\text{res}\langle\text{val}\rangle$.
4. Let valres be
 - the result of accessing IDX ROOT exp_i of val_b under C.
5. Result in valres .

2.2.15.3. Slice access expressions

Evaluating SLICE $\text{exp}_b \text{exp}_i \text{exp}_n$ under C:

1. Let $\text{res}\langle\text{val}\rangle$ be
 - the result of evaluating exp_b under C.
2. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$.
3. Let OK val_b be $\text{res}\langle\text{val}\rangle$.
4. Let valres be
 - the result of accessing SLICE ROOT $\text{exp}_i \text{exp}_n$ of val_b under C.

5. Result in **valres**.

2.2.16. Update expressions

Eval_path_upd:

- Updating **path** of **val** with **val** under **C**:
- Result in **res<val>**.

Updating ROOT of val_b with val_n under C:

1. Result in **OK val_n**.

Updating DOT path atom of val_b with val_n under C:

1. Let **res<val>** be
 - the result of **accessing path of val_b under C**.
2. Check that **res<val>** matches pattern **OK %**.
3. Let **OK val** be **res<val>**.
4. Check that **val** matches pattern **STR %**.
5. Let **STR valfield*** be **val**.
6. Let **atom_{field}*** be the list and **val_{field}*** be the list obtained by repeating:
 - Let **atom_{field} val_{field}** be **valfield**.for each **valfield** in **valfield***
7. Let **val'** be **val_n** if **atom** is equal to **atom_{field}** otherwise **val_{field}**.
8. Let **valfield_{new}*** be the list obtained by repeating:
 - Let **valfield_{new}** be **atom_{field} val'**.for each **atom_{field}** in **atom_{field}*** and **val_{field}** in **val_{field}***
9. Let **valres** be
 - the result of **updating path of val_b with STR valfield_{new}* under C**.
10. Result in **valres**.

Updating IDX path exp_i of val_b with val_n under C:

1. If let **TEXT t_n** be **val_n**:
 - a. Let **res<val>** be
 - the result of **accessing path of val_b under C**.

- b. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$.
 - c. Let OK val be $\text{res}\langle\text{val}\rangle$.
 - d. Check that val matches pattern $\text{TEXT } \%$.
 - e. Let $\text{TEXT } t$ be val .
 - f. Let $\text{res}\langle\text{val}\rangle'$ be
 - the result of **evaluating** exp_i **under** C .
 - g. Check that $\text{res}\langle\text{val}\rangle'$ matches pattern $\text{OK } \%$.
 - h. Let $\text{OK val}'$ be $\text{res}\langle\text{val}\rangle'$.
 - i. Check that val' has type num .
 - j. Let num be val' .
 - k. Check that num matches pattern $\text{NAT } \%$.
 - l. Let $\text{NAT } n_{\text{target}}$ be num .
 - m. Check that the length of t_n is equal to 1.
 - n. Let valres be
 - the result of **updating** path of val_b with $\text{TEXT } t[[n_{\text{target}}] = t_n]$ **under** C .
 - o. Result in valres .
2. Let $\text{res}\langle\text{val}\rangle$ be
 - the result of **accessing** path of val_b **under** C .
3. If let $\text{OK val}'$ be $\text{res}\langle\text{val}\rangle$:
 - a. Check that val' matches pattern $\text{LIST } \%$.
 - b. Let $\text{LIST val}'$ be val' .
 - c. Let $\text{res}\langle\text{val}\rangle'$ be
 - the result of **evaluating** exp_i **under** C .
 - d. Check that $\text{res}\langle\text{val}\rangle'$ matches pattern $\text{OK } \%$.
 - e. Let $\text{OK val}''$ be $\text{res}\langle\text{val}\rangle'$.
 - f. Check that val'' has type num .
 - g. Let num be val'' .
 - h. Check that num matches pattern $\text{NAT } \%$.
 - i. Let $\text{NAT } n_{\text{target}}$ be num .
 - j. Let valres be
 - the result of **updating** path of val_b with $\text{LIST val}'[[n_{\text{target}}] = \text{val}_n]$ **under** C .
 - k. Result in valres .

Updating SLICE path $\text{exp}_i \text{exp}_n$ of val_b with val_n **under** C :

1. If let $\text{TEXT } t_n$ be val_n :
 - a. Let $\text{res}\langle\text{val}\rangle$ be
 - the result of **accessing** path of val_b **under** C .

- b. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$.
- c. Let OK val be $\text{res}\langle\text{val}\rangle$.
- d. Check that val matches pattern $\text{TEXT } \%$.
- e. Let $\text{TEXT } t$ be val .
- f. Let $\text{res}\langle\text{val}\rangle'$ be
 - the result of **evaluating** exp_i **under** C .
- g. Check that $\text{res}\langle\text{val}\rangle'$ matches pattern $\text{OK } \%$.
- h. Let $\text{OK val}'$ be $\text{res}\langle\text{val}\rangle'$.
- i. Check that val' has type num .
- j. Let num be val' .
- k. Check that num matches pattern $\text{NAT } \%$.
- l. Let $\text{NAT } n_i$ be num .
- m. Let $\text{res}\langle\text{val}\rangle''$ be
 - the result of **evaluating** exp_n **under** C .
- n. Check that $\text{res}\langle\text{val}\rangle''$ matches pattern $\text{OK } \%$.
- o. Let $\text{OK val}''$ be $\text{res}\langle\text{val}\rangle''$.
- p. Check that val'' has type num .
- q. Let num' be val'' .
- r. Check that num' matches pattern $\text{NAT } \%$.
- s. Let $\text{NAT } n_n$ be num' .
- t. Check that the length of t_n is equal to n_n .
- u. Let valres be
 - the result of **updating** path of val_b with $\text{TEXT } t[[n_i : n_n] = t_n]$ **under** C .
- v. Result in valres .
2. Let $\text{res}\langle\text{val}\rangle$ be
 - the result of **accessing** path of val_b **under** C .
3. If let $\text{OK val}'$ be $\text{res}\langle\text{val}\rangle$:
 - a. Check that val' matches pattern $\text{LIST } \%$.
 - b. Let $\text{LIST val}'^*$ be val' .
 - c. Let $\text{res}\langle\text{val}\rangle'$ be
 - the result of **evaluating** exp_i **under** C .
 - d. Check that $\text{res}\langle\text{val}\rangle'$ matches pattern $\text{OK } \%$.
 - e. Let $\text{OK val}''$ be $\text{res}\langle\text{val}\rangle'$.
 - f. Check that val'' has type num .
 - g. Let num be val'' .
 - h. Check that num matches pattern $\text{NAT } \%$.
 - i. Let $\text{NAT } n_i$ be num .
 - j. Let $\text{res}\langle\text{val}\rangle''$ be

- the result of **evaluating** exp_n **under** C .
- k. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$.
- l. Let $\text{OK } \text{val}'$ be $\text{res}\langle\text{val}\rangle$.
- m. Check that val' has type num .
- n. Let num' be val' .
- o. Check that num' matches pattern $\text{NAT } \%$.
- p. Let $\text{NAT } n_n$ be num' .
- q. Let valres be
 - the result of **updating** path of val_b with $\text{LIST } \text{val}'[[n_i : n_n] = \text{val}_n]$ **under** C .
- r. Result in valres .

Evaluating $\text{UPD } \text{exp}_b \text{ path } \text{exp}_f$ **under** C :

1. Let $\text{res}\langle\text{val}\rangle$ be
 - the result of **evaluating** exp_b **under** C .
2. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$.
3. Let $\text{OK } \text{val}_b$ be $\text{res}\langle\text{val}\rangle$.
4. Let $\text{res}\langle\text{val}\rangle'$ be
 - the result of **evaluating** exp_f **under** C .
5. Check that $\text{res}\langle\text{val}\rangle'$ matches pattern $\text{OK } \%$.
6. Let $\text{OK } \text{val}_f$ be $\text{res}\langle\text{val}\rangle'$.
7. Let valres be
 - the result of **updating** path of val_b with val_f **under** C .
8. Result in valres .

2.2.17. Function call expressions

Evaluating $\text{CALL } \text{id } \text{targ}^* \text{arg}^*$ **under** C :

1. Let $\text{typ}_{\text{input}}^*$ be
 - the result of **evaluating** targ^* **under** C .
2. Let $\text{res}\langle\text{val}\rangle^*$ be the list obtained by repeating:
 - Let $\text{res}\langle\text{val}\rangle$ be
 - the result of **evaluating** arg **under** C .
 - for each arg in arg^*
3. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$, for all $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$.

4. Let $\text{val}_{\text{input}}^*$ be the list obtained by repeating:

- Let OK $\text{val}_{\text{input}}$ be $\text{res}\langle\text{val}\rangle$.

for each $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$

5. Let valres be

- the result of calling id with types targ^* and arguments $\text{val}_{\text{input}}^*$ under C .

6. Result in valres .

2.2.18. Iterator expressions

Evaluating ITER exp iterexp under C :

1. Let $\text{varr}^?$ be the iterated variable of ITER exp iterexp .

2. If let varr be $\text{varr}^?$:

a. Let val be ! the value of varr in C .

b. Result in OK val .

3. If the iterated variable of ITER exp iterexp is none:

a. Let iter vari^* be iterexp .

b. If iter is QUEST:

i. Let $C_{\text{sub}}^?$ be the sub-context of C for optional iterators vari^* .

ii. Let $\text{res}\langle\text{val}\rangle^*$ be the list obtained by repeating:

- Let $\text{res}\langle\text{val}\rangle$ be

- the result of evaluating exp under C_{sub} .

for each C_{sub} in C_{sub}^*

iii. Check that $\text{res}\langle\text{val}\rangle$ matches pattern OK %, for all $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$.

iv. Let val^* be the list obtained by repeating:

- Let OK val be $\text{res}\langle\text{val}\rangle$.

for each $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$

v. Result in OK OPT $\text{val}^?$.

c. Else:

i. Let C_{sub}^* be the sub-contexts of C for list iterators vari^* .

ii. Let $\text{res}\langle\text{val}\rangle^*$ be the list obtained by repeating:

- Let $\text{res}\langle\text{val}\rangle$ be

- the result of evaluating exp under C_{sub} .

for each C_{sub} in C_{sub}^*

iii. Check that $\text{res}\langle\text{val}\rangle$ matches pattern OK %, for all $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$.

iv. Let val^* be the list obtained by repeating:

- Let OK val be $res\langle val \rangle$.

for each $res\langle val \rangle$ in $res\langle val \rangle^*$

v. Result in OK LIST val^* .

2.3. Assignments

This section describes the semantics of assignments in the language. An assignment occurs when evaluating a let instruction, and when invoking a function or relation.

2.3.1. Assigning a value to an expression

On a let instruction and relation invocation, a value is assigned to an expression.

Assign_exp:

- Assigning val to exp under C :
- Result in C .

A sequence of assignments is defined as follows:

Assign_exps:

- Assigning each of val^* to each of exp^* under C :
- Result in C .

Assigning each of val^ to each of exp^* under C :*

1. If exp^* is an empty list:
 - a. Check that val^* is an empty list.
 - b. Result in C .
2. Else:
 - a. Let $exp_h :: exp_t^*$ be exp^* .
 - b. Check that val^* is a non-empty list.
 - c. Let $val_h :: val_t^*$ be val^* .
 - d. Let C' be
 - the result of assigning val_h to exp_h under C .
 - e. Let C'' be
 - the result of assigning each of val_t^* to each of exp_t^* under C' .
 - f. Result in C'' .

Only a subset of expressions can be assigned to.

2.3.1.1. Variable references

Assigning val''' to VAR id under C :

1. Let C' be C where $\text{id} \cdot \text{to val}'''$ added locally.
2. Result in C' .

2.3.1.2. Tuple expressions

2.3.1.3. Case expressions

Assigning INJ mixop val^* to INJ mixop exp^* under C :

1. Let C' be
 - the result of assigning each of val^* to each of exp^* under C .
2. Result in C' .

2.3.1.4. Struct expressions

Assigning STR valfield^* to STR expfield^* under C :

1. Let atom^* be the list and val^* be the list obtained by repeating:
 - Let atom val be valfield .for each valfield in valfield^*
2. Let atom'^* be the list and exp^* be the list obtained by repeating:
 - Let $\text{atom}' \text{ exp}$ be expfield .for each expfield in expfield^*
3. Check that the length of atom^* is equal to the length of atom'^* .
4. Check that atom' is equal to atom , for all atom in atom^* and atom' in atom'^* .
5. Let C' be
 - the result of assigning each of val^* to each of exp^* under C .
6. Result in C' .

2.3.1.5. Option expressions

Assigning $\text{OPT val}^?$ to $\text{OPT exp}^?$ under C :

1. If let exp be $\text{exp}^?$:
 - a. Check that $\text{val}^?$ is defined.
 - b. Let val be $\text{val}^?$.
 - c. Let C' be
 - the result of assigning val to exp under C .
 - d. Result in C' .
2. Else:
 - a. Check that $\text{val}^?$ is none.
 - b. Result in C .

2.3.1.6. List expressions

Assigning $\text{LIST } \text{val}^*$ to $\text{LIST } \text{exp}^*$ under C :

1. Let C' be
 - the result of assigning each of val^* to each of exp^* under C .
2. Result in C' .

2.3.1.7. Cons expressions

Assigning $\text{LIST } \text{val}_h :: \text{val}_t^*$ to $\text{CONS } \text{exp}_h \text{exp}_t$ under C :

1. Let C' be
 - the result of assigning val_h to exp_h under C .
2. Let C'' be
 - the result of assigning $\text{LIST } \text{val}_t^*$ to exp_t under C' .
3. Result in C'' .

2.3.1.8. Iterator expressions

Assigning val''' to $\text{ITER } \text{exp } \text{iterexp}$ under C :

1. Let $\text{varr}^?$ be the iterated variable of $\text{ITER } \text{exp } \text{iterexp}$.
2. If let varr be $\text{varr}^?$:
 - a. Let C' be C where varr to val''' added locally.
 - b. Result in C' .
3. If val''' is equal to OPT :
 - a. Check that the iterated variable of $\text{ITER } \text{exp } \text{iterexp}$ is none.
 - b. Let $\text{iter } \text{var}^*$ be iterexp .
 - c. Check that iter is QUEST .

- d. Let $\text{vari}_{\text{iter}}^*$ be the list obtained by repeating:
 - Let $\text{vari}_{\text{iter}}$ be vari iterated by QUEST.
 for each vari in vari^*
 - e. Let $\text{val}_{\text{bind}}^*$ be a list of the length of $\text{vari}_{\text{iter}}^*$ copies of OPT .
 - f. Let C' be C where each of $\text{vari}_{\text{iter}}^*$ to each of $\text{val}_{\text{bind}}^*$ added locally.
 - g. Result in C' .
4. If let $\text{OPT val}^?$ be val'' :
- a. Check that $\text{val}^?$ is defined.
 - b. Let val be $\text{val}^?$.
 - c. Check that the iterated variable of ITER exp iterexp is none.
 - d. Let iter vari^* be iterexp.
 - e. Check that iter is QUEST.
 - f. Let C' be
 - the result of assigning val to exp under C .
 - g. Let $\text{vari}_{\text{iter}}^*$ be the list obtained by repeating:
 - Let $\text{vari}_{\text{iter}}$ be vari iterated by QUEST.
 for each vari in vari^*
 - h. Let $\text{val}''^{?*}$ be the list obtained by repeating:
 - Let $\text{val}''^?$ be the value of vari in C' .
 for each vari in vari^*
 - i. Check that $\text{val}''^?$ is defined, for all $\text{val}''^?$ in $\text{val}''^{?*}$.
 - j. Let $\text{val}_{\text{sub}}^*$ be the list obtained by repeating:
 - Let val_{sub} be $\text{val}''^?$.
 for each $\text{val}''^?$ in $\text{val}''^{?*}$
 - k. Let C''^* be $(\text{\$add_varis}(C', \text{vari}_{\text{iter}}^*, \text{OPT val}_{\text{sub}}^*))^*$.
 - l. Check that C''^* is a list of length 1.
 - m. Let C'' be C''^* .
 - n. Result in C'' .
5. Else if let LIST val^* be val'' :
- a. Check that the iterated variable of ITER exp iterexp is none.
 - b. Let iter vari^* be iterexp.
 - c. Check that iter is STAR.
 - d. Let C_{local} be C with LOCAL.VAL set to $\text{\$empty_map}\langle \text{varr}, \text{val} \rangle$.
 - e. Let C_{sub}^* be the list obtained by repeating:

- Let C_{sub} be
 - the result of assigning val to exp under C_{local} .

for each val in val^*

f. Let $vari_{iter}^*$ be the list obtained by repeating:

- Let $vari_{iter}$ be $vari$ iterated by STAR.

for each $vari$ in $vari^*$

g. Let val'^{**} be the list obtained by repeating:

- Let val'^{**} be the value of $vari$ in each of C_{sub}^* .

for each $vari$ in $vari^*$

h. Check that val'^{**} is defined, for all val'^{**} in val'^{**} .

i. Let val_{sub}^{**} be the list obtained by repeating:

- Let val_{sub}^* be val'^{**} .

for each val'^{**} in val'^{**}

j. Let val_{bind}^* be the list obtained by repeating:

- Let val_{bind} be LIST val_{sub}^* .

for each val_{sub}^* in val_{sub}^{**}

k. Let C' be C where each of $vari_{iter}^*$ to each of val_{bind}^* added locally.

l. Result in C' .

2.3.2. Assigning a value to a function parameter

When invoking a function, the arguments are assigned to the parameters of the function. This is defined as follows:

Assign_param:

- Assigning $param$ to C under caller C and callee C :
- Result in C .

A sequence of parameter assignments is defined as follows:

Assign_params: $C_{callee} \setminus C \mid - param^* := val^* : \%$

Assigning each of $param^*$ to each of C under caller C and callee C_{callee} :

1. If $param^*$ is an empty list:

- a. Check that val^* is an empty list.
 - b. Result in C_{callee} .
2. Else:
- a. Let $param_h :: param_t^*$ be $param^*$.
 - b. Check that val^* is a non-empty list.
 - c. Let $val_h :: val_t^*$ be val^* .
 - d. Let $C_{callee'}$ be
 - the result of assigning $param_h$ to C under caller C and callee C_{callee} .
 - e. Let $C_{callee'} \setminus C \vdash param_t^* := val_t^* : C_{callee'}$.
 - f. Result in $C_{callee'}$.

2.3.2.1. Expression parameters

Assigning EXP typ exp to C under caller C and callee C_{callee} :

1. Let $C_{callee'}$ be
 - the result of assigning val to exp under C_{callee} .
2. Result in $C_{callee'}$.

2.3.2.2. Function parameters

Assigning FUN $id_{callee} : tparam^* param^* \rightarrow typ$ to C under caller C and callee C_{callee} :

1. Let $(_ funcdef)$ be ! the function definition of id_{caller} in C .
2. Let $C_{callee'}$ be C_{callee} where function id_{callee} to $funcdef$ added locally.
3. Result in $C_{callee'}$.

2.4. Instructions

A function and relation body is defined as a sequence of instructions.

Eval_instr:

- Evaluating $instr$ under C :
- Result in $flow$.

Evaluating an instruction yields a $flow$ as a result, representing the control flow.

```
flow
: OK
| RES val*
| RET val
| FAIL
```

;

If none of the below rules apply, the result is a **FAIL**,

A sequence of instructions is evaluated as follows:

Eval_instrs:

- Evaluating **instr^{*}** under **C**:
- Result in **flow**.

Evaluating instr^{*} under C:

1. If **instr^{*}** is an empty list:
 - a. Result in **OK**.
2. Else:
 - a. Let **instr_h :: instr_t^{*}** be **instr^{*}**.
 - b. Let **flow'** be
 - the result of **evaluating instr_h under C**.
 - c. If let **RES val^{*}** be **flow'**:
 - i. Result in **RES val^{*}**.
 - d. Else if let **RET val** be **flow'**:
 - i. Result in **RET val**.
 - e. Else if **flow'** is **OK**:
 - i. Let **flow** be
 - the result of **evaluating instr_t^{*} under C**.
 - ii. Result in **flow**.
 - f. Else:
 - i. Let **flow** be
 - the result of **evaluating instr_t^{*} under C**.
 - ii. Result in **flow**.

A block is a sequence of instructions, and is evaluated as follows:

Eval_block:

- Evaluating **block** under **C**:
- Result in **flow**.

Evaluating block under C:

1. Let flow be
 - the result of evaluating block under C.
2. Result in flow.

2.4.1. If instructions

Evaluating IF exp iterexp* block under C:

1. Let iterexp_{rev}* be the reversal of iterexp*.
2. If evaluaing condition exp with iteration iterexp_{rev}* under C yields true:
 - a. Let flow be
 - the result of evaluating block under C.
 - b. Result in flow.
3. Else:
 - a. Result in FAIL.

An if condition may be iterated, where the condition must evaluate to true for all sub-contexts of the iteration.

Eval_if_cond:

- Evaluaing condition exp with iteration iterexp* under C yields true

Evaluaing condition exp with iteration iterexp* under C yields true when:

1. If iterexp* is an empty list:
 - a. Let res<val> be
 - the result of evaluating exp under C.
 - b. Check that res<val> is equal to OK BOOL true.
 - c. Then, the relation holds.
2. Else:
 - a. Let iterexp_h :: iterexp_t* be iterexp*.
 - b. Let iter vari* be iterexp_h.
 - c. If iter is QUEST:
 - i. Let C_{sub}? be the sub-context of C for optional iterators vari*.
 - ii. Check that evaluaing condition exp with iteration iterexp_t* under C_{sub} yields true, for all C_{sub} in C_{sub}*.
 - iii. Then, the relation holds.
 - d. Else:

- i. Let C_{sub}^* be the sub-contexts of C for list iterators $vari^*$.
- ii. Check that evaluating condition exp with iteration $iterexp_t^*$ under C_{sub} yields true, for all C_{sub} in C_{sub}^* .
- iii. Then, the relation holds.

2.4.2. If-hold instructions

A relation without an output may be used as a condition.

Evaluating IFHOLD id with inputs exp^* $iterexp^*$ holdcase under C :

1. Let $iterexp_{rev}^*$ be the reversal of $iterexp^*$.
2. If evaluating relation id with inputs exp^* and iteration $iterexp_{rev}^*$ under C holds:
 - a. If let BOTH $block_{hold}$ $block_{nothold}$ be holdcase:
 - i. Let $flow$ be
 - the result of evaluating $block_{hold}$ under C .
 - ii. Result in $flow$.
 - b. Else:
 - i. Let HOLD $block_{hold}$ be holdcase.
 - ii. Let $flow$ be
 - the result of evaluating $block_{hold}$ under C .
 - iii. Result in $flow$.
3. Else:
 - a. If let BOTH $block_{hold}$ $block_{nothold}$ be holdcase:
 - i. Let $flow$ be
 - the result of evaluating $block_{nothold}$ under C .
 - ii. Result in $flow$.
 - b. Else:
 - i. Let NOTHOLD $block_{nothold}$ be holdcase.
 - ii. Let $flow$ be
 - the result of evaluating $block_{nothold}$ under C .
 - iii. Result in $flow$.

The condition is evaluated by evaluating the relation body for all sub-contexts of the iteration, and checking that the relation holds for all sub-contexts.

Eval_ifhold_cond:

- Evaluating relation id with inputs exp^* and iteration $iterexp^*$ under C holds

Evaluating relation id with inputs exp^* and iteration $iterexp^*$ under C holds when:

1. If iterexp^* is an empty list:
 - a. Let $\text{res}\langle\text{val}\rangle^*$ be the list obtained by repeating:
 - Let $\text{res}\langle\text{val}\rangle$ be
 - the result of evaluating exp under C .
 for each exp in exp^*
 - b. Check that $\text{res}\langle\text{val}\rangle$ matches pattern $\text{OK } \%$, for all $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$.
 - c. Let val^* be the list obtained by repeating:
 - Let OK val be $\text{res}\langle\text{val}\rangle$.
 for each $\text{res}\langle\text{val}\rangle$ in $\text{res}\langle\text{val}\rangle^*$
 - d. Let $\text{res}\langle\text{val}^*\rangle$ be
 - the result of calling id with arguments val^* under C .
 - e. Check that $\text{res}\langle\text{val}^*\rangle$ is equal to $\text{OK } \cdot$.
 - f. Then, the relation holds.
2. Else:
 - a. Let $\text{iterexp}_h :: \text{iterexp}_t^*$ be iterexp^* .
 - b. Let iter vari^* be iterexp_h .
 - c. If iter is QUEST :
 - i. Let $C_{\text{sub}}^?$ be the sub-context of C for optional iterators vari^* .
 - ii. Check that evaluating relation id with inputs exp^* and iteration iterexp_t^* under C_{sub} holds, for all C_{sub} in C_{sub}^* .
 - iii. Then, the relation holds.
 - d. Else:
 - i. Let C_{sub}^* be the sub-contexts of C for list iterators vari^* .
 - ii. Check that evaluating relation id with inputs exp^* and iteration iterexp_t^* under C_{sub} holds, for all C_{sub} in C_{sub}^* .
 - iii. Then, the relation holds.

2.4.3. Case instructions

Evaluating CASE exp case^* under C :

1. Let $\text{res}\langle\text{block}\rangle$ be
 - the result of evaluating case matches for exp against case^* under C .
2. Check that $\text{res}\langle\text{block}\rangle$ matches pattern $\text{OK } \%$.
3. Let OK block be $\text{res}\langle\text{block}\rangle$.
4. Let flow be
 - the result of evaluating block under C .

5. Result in **flow**.

A matching case is selected as follows:

Eval_cases:

- Evaluating case matches for **exp** against **case*** under **C**:
- Result in **res<block>**.

Evaluating case matches for **exp** against **case*** under **C**:

1. If **case*** is an empty list:
 - a. Result in **FAIL**.
2. Else:
 - a. Let $\text{case}_h :: \text{case}_t^*$ be **case***.
 - b. Let **res<block>** be
 - the result of **evaluating case match for exp against case_h under C**.
 - c. If let **OK block** be **res<block>**:
 - i. Result in **OK block**.
 - d. Else:
 - i. Let **blockres** be
 - the result of **evaluating case matches for exp against case_t* under C**.
 - ii. Result in **blockres**.

Eval_case:

- Evaluating case match for **exp** against **case** under **C**:
- Result in **res<block>**.

Evaluating case match for **exp** against **guard block** under **C**:

1. Let exp_{cond} be **guard** as an expression matching against **exp**.
2. Let **res<val>** be
 - the result of **evaluating exp_{cond} under C**.
3. Check that **res<val>** is equal to **OK BOOL true**.
4. Result in **OK block**.

2.4.4. Group instructions

A group is simply a wrapper around a sequence of instructions, used as an intermediary for prose generation backend.

Evaluating $\text{GROUP id exp}^* \text{ : typ}^* \text{ } \rightarrow \text{ typ}^* \text{ } = \text{block under C}$:

1. Let flow be
 - the result of evaluating block under C .
2. Result in flow .

2.4.5. Let instructions

Evaluating $\text{LET exp}_i \text{ } = \text{exp}_r \text{ iterinstr}^* \text{ block under C}$:

1. Let res<ctx> be
 - the result of evaluating assignment $\text{exp}_i \text{ } = \text{exp}_r \text{ with iteration iterinstr}^* \text{ under C}$.
2. Check that res<ctx> matches pattern $\text{OK \%}$.
3. Let OK C' be res<ctx> .
4. Let flow be
 - the result of evaluating block under C' .
5. Result in flow .

A let assignment may also be iterated. Iteration dimensions of the right-hand side are evaluated outside-in, reaching the innermost assignment, then the assigned contexts are collected inside-out to produce the final result.

Eval_let:

- Evaluating assignment $\text{exp} \text{ } = \text{exp}$ with iteration iterinstr^* under C :
- Result in ctxres .

Evaluating assignment $\text{exp}_i \text{ } = \text{exp}_r \text{ with iteration iterinstr}^* \text{ under C}$:

1. If iterinstr^* is an empty list:
 - a. Let res<val> be
 - the result of evaluating $\text{exp}_r \text{ under C}$.
 - b. Check that res<val> matches pattern $\text{OK \%}$.
 - c. Let OK val_i be res<val> .
 - d. Let C' be
 - the result of assigning $\text{val}_i \text{ to exp}_i \text{ under C}$.
 - e. Result in OK C' .

2. Else:

- a. Let $\text{iterinstr}_h :: \text{iterinstr}_t^*$ be iterinstr^* .
- b. Let $\text{iter vari}_{\text{bound}}^* \text{ vari}_{\text{bind}}^*$ be iterinstr_h .
- c. If iter is QUEST:
 - i. If the sub-context of C for optional iterators $\text{vari}_{\text{bound}}^*$ is none:
 - A. Let $\text{vari}_{\text{bind_iter}}^*$ be the list obtained by repeating:
 - Let $\text{vari}_{\text{bind_iter}}$ be $\text{vari}_{\text{bind}}$ iterated by QUEST.
 for each $\text{vari}_{\text{bind}}$ in $\text{vari}_{\text{bind}}^*$
 - B. Let $\text{val}_{\text{bind}}^*$ be a list of the length of $\text{vari}_{\text{bind}}^*$ copies of $\text{OPT } \cdot$.
 - C. Let C_{res} be C where each of $\text{vari}_{\text{bind_iter}}^*$ to each of $\text{val}_{\text{bind}}^*$ added locally.
 - D. Result in $\text{OK } C_{\text{res}}$.
 - ii. Let $C^?$ be the sub-context of C for optional iterators $\text{vari}_{\text{bound}}^*$.
 - iii. If let C_{sub} be $C^?$:
 - A. Let $\text{res} < \text{ctx} >$ be
 - the result of evaluating assignment $\text{exp}_l \leftarrow \text{exp}_r$ with iteration iterinstr_t^* under C_{sub} .
 - B. Check that $\text{res} < \text{ctx} >$ matches pattern $\text{OK } \%$.
 - C. Let $\text{OK } C_{\text{sub_res}}$ be $\text{res} < \text{ctx} >$.
 - D. Let $\text{vari}_{\text{bind_iter}}^*$ be the list obtained by repeating:
 - Let $\text{vari}_{\text{bind_iter}}$ be $\text{vari}_{\text{bind}}$ iterated by QUEST.
 for each $\text{vari}_{\text{bind}}$ in $\text{vari}_{\text{bind}}^*$
 - E. Let $\text{val}^{?*}$ be the list obtained by repeating:
 - Let $\text{val}^?$ be the value of $\text{vari}_{\text{bind}}$ in $C_{\text{sub_res}}$
 for each $\text{vari}_{\text{bind}}$ in $\text{vari}_{\text{bind}}^*$
 - F. Check that $\text{val}^?$ is defined, for all $\text{val}^?$ in $\text{val}^{?*}$.
 - G. Let $\text{val}_{\text{bind}}^*$ be the list obtained by repeating:
 - Let val_{bind} be $\text{val}^?$.
 for each $\text{val}^?$ in $\text{val}^{?*}$
 - H. Let C_{res} be C where each of $\text{vari}_{\text{bind_iter}}^*$ to each of $(\text{OPT } \text{val}_{\text{bind}})^*$ added locally.
 - I. Result in $\text{OK } C_{\text{res}}$.
- d. Else:
 - i. If the sub-contexts of C for list iterators $\text{vari}_{\text{bound}}^*$ is an empty list:
 - A. Let $\text{vari}_{\text{bind_iter}}^*$ be the list obtained by repeating:
 - Let $\text{vari}_{\text{bind_iter}}$ be $\text{vari}_{\text{bind}}$ iterated by STAR.
 for each $\text{vari}_{\text{bind}}$ in $\text{vari}_{\text{bind}}^*$

- B. Let $\text{val}_{\text{bind}}^*$ be a list of the length of $\text{vari}_{\text{bind}}^*$ copies of LIST .
- C. Let C_{res} be C where each of $\text{vari}_{\text{bind_iter}}^*$ to each of $\text{val}_{\text{bind}}^*$ added locally.
- D. Result in $\text{OK } \text{C}_{\text{res}}$.
- ii. Let C_{sub}^* be the sub-contexts of C for list iterators $\text{vari}_{\text{bound}}^*$.
- iii. If C_{sub}^* is a non-empty list:
- A. Let $\text{res}<\text{ctx}>^*$ be the list obtained by repeating:
- Let $\text{res}<\text{ctx}>$ be
 - the result of evaluating assignment $\text{exp}_l := \text{exp}_r$ with iteration iterinstr_t^* under C_{sub} .
- for each C_{sub} in C_{sub}^*
- B. Check that $\text{res}<\text{ctx}>$ matches pattern $\text{OK } \%$, for all $\text{res}<\text{ctx}>$ in $\text{res}<\text{ctx}>^*$.
- C. Let $\text{C}_{\text{sub_res}}^*$ be the list obtained by repeating:
- Let $\text{OK } \text{C}_{\text{sub_res}}$ be $\text{res}<\text{ctx}>$.
- for each $\text{res}<\text{ctx}>$ in $\text{res}<\text{ctx}>^*$
- D. Let $\text{val}^{*?}$ be the list obtained by repeating:
- Let $\text{val}^{*?}$ be the values of $\text{vari}_{\text{bind}}^*$ in $\text{C}_{\text{sub_res}}$.
- for each $\text{C}_{\text{sub_res}}$ in $\text{C}_{\text{sub_res}}^*$
- E. Check that $\text{val}^{*?}$ is defined, for all $\text{val}^{*?}$ in $\text{val}^{*?}$.
- F. Let $\text{val}_{\text{sub_bind}}^{**}$ be the list obtained by repeating:
- Let $\text{val}_{\text{sub_bind}}^{*?}$ be $\text{val}^{*?}$.
- for each $\text{val}^{*?}$ in $\text{val}^{*?}$
- G. Let $\text{val}_{\text{bind}}^{**}$ be $\$ \text{transpose_} <\text{val}> (\text{val}_{\text{sub_bind}}^{**})$.
- H. Let $\text{val}_{\text{res_bind}}^*$ be the list obtained by repeating:
- Let $\text{val}_{\text{res_bind}}$ be $\text{LIST } \text{val}_{\text{bind}}^*$.
- for each $\text{val}_{\text{bind}}^*$ in $\text{val}_{\text{bind}}^{**}$
- I. Let $\text{vari}_{\text{bind_iter}}^*$ be the list obtained by repeating:
- Let $\text{vari}_{\text{bind_iter}}$ be $\text{vari}_{\text{bind}}$ iterated by STAR .
- for each $\text{vari}_{\text{bind}}$ in $\text{vari}_{\text{bind}}^*$
- J. Let C_{res} be C where each of $\text{vari}_{\text{bind_iter}}^*$ to each of $\text{val}_{\text{res_bind}}^*$ added locally.
- K. Result in $\text{OK } \text{C}_{\text{res}}$.

2.4.6. Relation call instructions

Evaluating REL id `: exp_{in}^{*} -> exp_{out}^{*} iterinstr^{*} block under C:

1. Let res<ctx> be
 - the result of evaluating relation id with inputs exp_{in}^{*}, outputs exp_{out}^{*}, and iteration iterinstr^{*} under C.
2. Check that res<ctx> matches pattern OK %.
3. Let OK C' be res<ctx>.
4. Let flow be
 - the result of evaluating block under C'.
5. Result in flow.

A relation call instruction is a mixture of a relation call and an assignment.

Eval_rel:

- Evaluating relation id with inputs exp^{*}, outputs exp^{*}, and iteration iterinstr^{*} under C:
- Result in ctxres.

Evaluating relation id with inputs exp_{in}^{*}, outputs exp_{out}^{*}, and iteration iterinstr^{*} under C:

1. If iterinstr^{*} is an empty list:
 - a. Let res<val>^{*} be the list obtained by repeating:
 - Let res<val> be
 - the result of evaluating exp_{in} under C.for each exp_{in} in exp_{in}^{*}
 - b. Check that res<val> matches pattern OK %, for all res<val> in res<val>^{*}.
 - c. Let val_{in}^{*} be the list obtained by repeating:
 - Let OK val_{in} be res<val>.for each res<val> in res<val>^{*}
 - d. Let res<val*> be
 - the result of calling id with arguments val_{in}^{*} under C.
 - e. Check that res<val*> matches pattern OK %.
 - f. Let OK val_{out}^{*} be res<val*>.
 - g. Let C' be
 - the result of assigning each of val_{out}^{*} to each of exp_{out}^{*} under C.
 - h. Result in OK C'.

2. Else:

- a. Let $\text{iterinstr}_h :: \text{iterinstr}_t^*$ be iterinstr^* .
- b. Let $\text{iter vari}_{\text{bound}}^* \text{ vari}_{\text{bind}}^*$ be iterinstr_h .
- c. If iter is QUEST:
 - i. If the sub-context of C for optional iterators $\text{vari}_{\text{bound}}^*$ is none:
 - A. Let $\text{vari}_{\text{bind_iter}}^*$ be the list obtained by repeating:
 - Let $\text{vari}_{\text{bind_iter}}$ be $\text{vari}_{\text{bind}}$ iterated by QUEST.
 for each $\text{vari}_{\text{bind}}$ in $\text{vari}_{\text{bind}}^*$
 - B. Let $\text{val}_{\text{bind}}^*$ be a list of the length of $\text{vari}_{\text{bind}}^*$ copies of OPT .
 - C. Let C_{res} be C where each of $\text{vari}_{\text{bind_iter}}^*$ to each of $\text{val}_{\text{bind}}^*$ added locally.
 - D. Result in $\text{OK } C_{\text{res}}$.
 - ii. Let $C^?$ be the sub-context of C for optional iterators $\text{vari}_{\text{bound}}^*$.
 - iii. If let C_{sub} be $C^?$:
 - A. Let $\text{res}<\text{ctx}>$ be
 - the result of evaluating relation id with inputs exp_{in}^* , outputs $\text{exp}_{\text{out}}^*$, and iteration iterinstr_t^* under C_{sub} .
 - B. Check that $\text{res}<\text{ctx}>$ matches pattern $\text{OK } \%$.
 - C. Let $\text{OK } C_{\text{sub_res}}$ be $\text{res}<\text{ctx}>$.
 - D. Let $\text{vari}_{\text{bind_iter}}^*$ be the list obtained by repeating:
 - Let $\text{vari}_{\text{bind_iter}}$ be $\text{vari}_{\text{bind}}$ iterated by QUEST.
 for each $\text{vari}_{\text{bind}}$ in $\text{vari}_{\text{bind}}^*$
 - E. Let $\text{val}^{?*$ be the list obtained by repeating:
 - Let $\text{val}^?$ be the value of $\text{vari}_{\text{bind}}$ in $C_{\text{sub_res}}$
 for each $\text{vari}_{\text{bind}}$ in $\text{vari}_{\text{bind}}^*$
 - F. Check that $\text{val}^?$ is defined, for all $\text{val}^?$ in $\text{val}^{?*$.
 - G. Let $\text{val}_{\text{bind}}^*$ be the list obtained by repeating:
 - Let val_{bind} be $\text{val}^?$.
 for each $\text{val}^?$ in $\text{val}^{?*$
 - H. Let C_{res} be C where each of $\text{vari}_{\text{bind_iter}}^*$ to each of $(\text{OPT } \text{val}_{\text{bind}})^*$ added locally.
 - I. Result in $\text{OK } C_{\text{res}}$.
- d. Else:
 - i. If the sub-contexts of C for list iterators $\text{vari}_{\text{bound}}^*$ is an empty list:
 - A. Let $\text{vari}_{\text{bind_iter}}^*$ be the list obtained by repeating:
 - Let $\text{vari}_{\text{bind_iter}}$ be $\text{vari}_{\text{bind}}$ iterated by STAR.

for each $\text{vari}_{\text{bind}}$ in $\text{vari}_{\text{bind}}^*$

B. Let $\text{val}_{\text{bind}}^*$ be a list of the length of $\text{vari}_{\text{bind}}^*$ copies of LIST .

C. Let C_{res} be C where each of $\text{vari}_{\text{bind_iter}}^*$ to each of $\text{val}_{\text{bind}}^*$ added locally.

D. Result in OK C_{res} .

ii. Let C_{sub}^* be the sub-contexts of C for list iterators $\text{vari}_{\text{bound}}^*$.

iii. If C_{sub}^* is a non-empty list:

A. Let $\text{res}<\text{ctx}>^*$ be the list obtained by repeating:

- Let $\text{res}<\text{ctx}>$ be
 - the result of evaluating relation id with inputs exp_{in}^* , outputs $\text{exp}_{\text{out}}^*$, and iteration iterinstr_t^* under C_{sub} .

for each C_{sub} in C_{sub}^*

B. Check that $\text{res}<\text{ctx}>$ matches pattern $\text{OK } \%$, for all $\text{res}<\text{ctx}>$ in $\text{res}<\text{ctx}>^*$.

C. Let $\text{C}_{\text{sub_res}}^*$ be the list obtained by repeating:

- Let $\text{OK C}_{\text{sub_res}}$ be $\text{res}<\text{ctx}>$.

for each $\text{res}<\text{ctx}>$ in $\text{res}<\text{ctx}>^*$

D. Let $\text{val}^{?*}$ be the list obtained by repeating:

- Let $\text{val}^{*?}$ be the values of $\text{vari}_{\text{bind}}^*$ in $\text{C}_{\text{sub_res}}$.

for each $\text{C}_{\text{sub_res}}$ in $\text{C}_{\text{sub_res}}^*$

E. Check that $\text{val}^{*?}$ is defined, for all $\text{val}^{*?}$ in $\text{val}^{?*}$.

F. Let $\text{val}_{\text{sub_bind}}^{**}$ be the list obtained by repeating:

- Let $\text{val}_{\text{sub_bind}}^{*}$ be $\text{val}^{*?}$.

for each $\text{val}^{*?}$ in $\text{val}^{?*}$

G. Let $\text{val}_{\text{bind}}^{**}$ be $\text{\$transpose_<val>(\text{val}_{\text{sub_bind}}^{**})}$.

H. Let $\text{val}_{\text{res_bind}}^*$ be the list obtained by repeating:

- Let $\text{val}_{\text{res_bind}}$ be $\text{LIST val}_{\text{bind}}^*$.

for each $\text{val}_{\text{bind}}^*$ in $\text{val}_{\text{bind}}^{**}$

I. Let $\text{vari}_{\text{bind_iter}}^*$ be the list obtained by repeating:

- Let $\text{vari}_{\text{bind_iter}}$ be $\text{vari}_{\text{bind}}$ iterated by STAR .

for each $\text{vari}_{\text{bind}}$ in $\text{vari}_{\text{bind}}^*$

J. Let C_{res} be C where each of $\text{vari}_{\text{bind_iter}}^*$ to each of $\text{val}_{\text{res_bind}}^*$ added locally.

K. Result in OK C_{res} .

2.4.7. Result instructions

Evaluating `RESULT typ* `-> typ* `: exp*` under `C`:

1. Let `res<val>*` be the list obtained by repeating:
 - Let `res<val>` be
 - the result of evaluating `exp` under `C`.for each `exp` in `exp*`
2. Check that `res<val>` matches pattern `OK %`, for all `res<val>` in `res<val>*`.
3. Let `val*` be the list obtained by repeating:
 - Let `OK val` be `res<val>`.for each `res<val>` in `res<val>*`
4. Result in `RES val*`.

2.4.8. Return instructions

Evaluating `RETURN exp` under `C`:

1. Let `res<val>` be
 - the result of evaluating `exp` under `C`.
2. Check that `res<val>` matches pattern `OK %`.
3. Let `OK val` be `res<val>`.
4. Result in `RET val`.

2.5. Functions

An entry point for function calls is the following relation:

Call_func:

- Calling `id` with types `targ*` and arguments `val*` under `C`:
- Result in `res<val>`.

Calling `id` with types `typ*` and arguments `val*` under `C`:

1. Let `funccache` be meta-cache entry for function `id` with inputs `val*`.

2. Let $C \vdash \text{id typ}^* \text{val}^* \text{funccache} : \text{valres}$.
3. Result in valres .

A meta-cache is used for performance reasons.

Call_func_cache: $C \vdash \text{id typ}^* \text{val}^* \text{funccache} : \%$

$C \vdash \text{id typ}^* \text{val}^* \text{funccache} : \%$

1. If let $\text{OK val}_{\text{cached}}$ be funccache :
 - a. Result in $\text{OK val}_{\text{cached}}$.
2. Else:
 - a. Let $(\text{cursor}, \text{funcdef})$ be ! the function definition of id in C .
 - b. Let $\text{cp}_{\text{before}}$ be cache checkpoint.
 - c. Let $C \vdash \text{funcdef typ}^* \text{val}^* : \text{valres}$.
 - d. Let cp_{after} be cache checkpoint.
 - e. Let b_{ext} be funcdef has type externFuncDef .
 - f. Let b_{seff} be side-effect occurred between checkpoints $\text{cp}_{\text{before}}$ and cp_{after} .
 - g. Let b_{anon} be cursor is equal to LOCAL .
 - h. Let bool^* be the list obtained by repeating:
 - Let bool be val is a function.
 for each val in val^*
 - i. Let b_{hoff} be some of bool^* is true.
 - j. Check that $\$ \text{cache_add_func_maybe}(\text{b}_{\text{ext}} \vee \text{b}_{\text{seff}} \vee \text{b}_{\text{anon}} \vee \text{b}_{\text{hoff}}, \text{id}, \text{val}^*, \text{valres})$.
 - k. Result in valres .

The actual function call is performed by the following relation, dispatching according to the function type:

Call_func_dispatch: $C \vdash \text{funcdef typ}^* \text{val}^* : \%$

$C \vdash \text{funcdef typ}^* \text{val}^* : \%$

1. If let EXT id be funcdef :
 - a. Let valres be
 - the result of evaluating extern function id with types typ^* and inputs val^* .
 - b. Result in valres .
2. Else if let builtinFuncDef be funcdef :

- a. Let `BUILTIN id _ _` be `builtinFuncDef`.
 - b. Let `| - id '@ '< typ* > `(val*) : valres`.
 - c. Result in `valres`.
3. Else if let `tableFuncDef` be `funcdef`:
 - a. Check that `typ*` is an empty list.
 - b. Let `valres` be
 - the result of `evaluating tableFuncDef with inputs val*` under `C`.
 - c. Result in `valres`.
 4. Else:
 - a. Let `definedFuncDef` be `funcdef`.
 - b. Let `valres` be
 - the result of `evaluating definedFuncDef with types typ* and inputs val*` under `C`.
 - c. Result in `valres`.

2.5.1. Extern functions

`Call_extern_func`:

- Evaluating extern function `id` with types `typ*` and inputs `val*`:
- Result in `res<val>`.

2.5.2. Built-in functions

`Call_builtin_func`: `| - id '@ '< typ* > `(val*) : %`

2.5.3. Table functions

`Call_table_func`:

- Evaluating `tableFuncDef` with inputs `val*` under `C`:
- Result in `res<val>`.

`Evaluating tableFuncDef with inputs val* under C`:

1. Let `TABLE param* -> tblrow*` be `tableFuncDef`.
2. Let `Ccallee` be `C` with `LOCAL` set to `$empty_layer`.
3. Let `Ccallee \ C | - param* := val* : Ccallee`.
4. Let `valres` be
 - the result of `evaluating tblrow* with inputs val*` under `Ccallee`.

5. Result in **valres**.

Table rows are evaluated as follows:

Eval_tblrows:

- Evaluating **tblrow*** with inputs **val*** under **C**:
- Result in **res<val>**.

Evaluating tblrow* with inputs val* under C:

1. If **tblrow*** is an empty list:
 - a. Result in **FAIL**.
2. Else:
 - a. Let **tblrow_n :: tblrow_t*** be **tblrow***.
 - b. Let **res<val>** be
 - the result of **evaluating tblrow_n with inputs val* under C**.
 - c. If let **OK val_{out}** be **res<val>**:
 - i. Result in **OK val_{out}***.
 - d. Else:
 - i. Let **valres** be
 - the result of **evaluating tblrow_t* with inputs val* under C**.
 - ii. Result in **valres**.

Eval_tblrow:

- Evaluating **tblrow** with inputs **val*** under **C**:
- Result in **res<val>**.

Evaluating tblrow with inputs val* under C:

1. Let **_` = exp<sub>out</sub>` - block** be **tblrow**.
2. Let **flow** be
 - the result of **evaluating block under C**.
3. Check that **flow** matches pattern **RET %**.
4. Let **RET val_{out}** be **flow**.
5. Result in **OK val_{out}***.

2.5.4. User-defined functions

Call_defined_func:

- Evaluating `definedFuncDef` with types `typ*` and inputs `val*` under `C`:
- Result in `res<val>`.

Evaluating `definedFuncDef` with types `typ*` and inputs `val*` under `C`:

1. Let `DEF tparam* param* -> block blockels?` be `definedFuncDef`.
2. Check that the length of `tparam*` is equal to the length of `typ*`.
3. Let `tdenv` be `{ ( tparam : DEF · -> ALIAS typ ) }`.
4. Let `Ccallee` be `C` with `LOCAL` set to `$empty_layer[TYP = tdenv]`.
5. Let `Ccallee / C | - param* := val* : Ccallee`.
6. Let `block'` be `blockels?` as a block.
7. Let `blockall` be `block` concatenated with `block'`.
8. Let `flow` be
 - the result of evaluating `blockall` under `Ccallee`.
9. Check that `flow` matches pattern `RET %`.
10. Let `RET valret` be `flow`.
11. Result in `OK valret`.

2.6. Relations

An entry point for relation calls is the following relation:

Call_rel:

- Calling `id` with arguments `val*` under `C`:
- Result in `res<val*>`.

Calling `id` with arguments `val*` under `C`:

1. Let `relcache` be meta-cache entry for relation `id` with inputs `val*`.
2. Let `C | - id val* relcache : valsres`.
3. Result in `valsres`.

A meta-cache is used for performance reasons.

Call_rel_cache: $C \mid - \text{id } \text{val}_{\text{input}}^* \text{ relcache} : \%$

$C \mid - \text{id } \text{val}_{\text{input}}^* \text{ relcache} : \%$:

1. If let OK $\text{val}_{\text{cached}}^*$ be relcache:
 - a. Result in OK $\text{val}_{\text{cached}}^*$.
2. Else:
 - a. Let reldef be ! the relation definition of id in C.
 - b. Let $\text{cp}_{\text{before}}$ be cache checkpoint.
 - c. Let $C \mid - \text{reldef } \text{val}_{\text{input}}^* : \text{valsres}$.
 - d. Let cp_{after} be cache checkpoint.
 - e. Let b_{ext} be reldef has type externRelDef.
 - f. Let b_{seff} be side-effect occurred between checkpoints $\text{cp}_{\text{before}}$ and cp_{after} .
 - g. Check that $\$cache_add_rel_maybe(\text{b}_{\text{ext}} \vee \text{b}_{\text{seff}}, \text{id}, \text{val}_{\text{input}}^*, \text{valsres})$.
 - h. Result in valsres.

The actual function call is performed by the following relation, dispatching according to the function type:

Call_rel_dispatch: $C \mid - \text{reldef } \text{val}^* : \%$

$C \mid - \text{reldef } \text{val}^* : \%$:

1. If let EXT id be reldef:
 - a. Let valsres be
 - the result of evaluating extern relation id with inputs val^* .
 - b. Result in valsres.
2. Else:
 - a. Let definedRelDef be reldef.
 - b. Let valsres be
 - the result of evaluating definedRelDef with inputs val^* under C.
 - c. Result in valsres.

2.6.1. Extern relations

Call_extern_rel:

- Evaluating extern relation id with inputs val^* :
- Result in $\text{res} \langle \text{val}^* \rangle$.

2.6.2. User-defined relations

Call_defined_rel:

- Evaluating `definedRelDef` with inputs `val*` under `C`:
- Result in `res<val*>`.

Evaluating `definedRelDef` with inputs `val*` under `C`:

1. Let `DEF exp* block blockels?` be `definedRelDef`.
2. Let `Clocal` be `C` with `LOCAL` set to `$empty_layer`.
3. Let `Clocal'` be
 - the result of assigning each of `val*` to each of `exp*` under `Clocal`.
4. Let `block'` be `blockels?` as a block.
5. Let `blockall` be `block` concatenated with `block'`.
6. Let `flow` be
 - the result of evaluating `blockall` under `Clocal'`.
7. Check that `flow` matches pattern `RES %`.
8. Let `RES valout*` be `flow`.
9. Result in `OK valout*`.